Team 3: Presentation 2

Ronan Howard, Jingyu Huo, Yifan Xia, Taylor Massey

Coders

Yifan Xia & Jingyu Huo

KimiaNet vs UNI2

- **Unchanged MIL architecture:** Same hierarchical attention head and original variable-sized patches; no cropping to 224×224.
- **Encoder replacement:** KimiaNet → UNI2-h, with padding to make patch height and width divisible by 14.
- **Training:** Frozen encoder; trainable 1536→512 projection, attention modules, and classifier.
- **Controlled comparison:** Same data splits, cases, and maximum number of patches per case.

KimiaNet vs UNI2 results

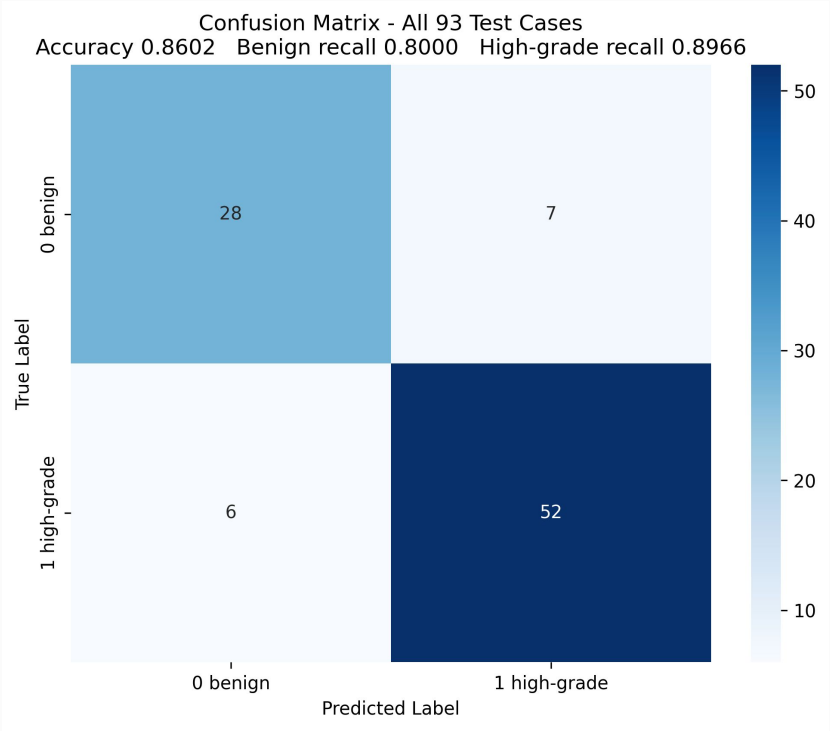
KimiaNet	Accuracy	Benign recall	High-grade recall
Split 01	0.9444	0.8571	1.0000
Split 02	0.7368	0.5714	0.8333
Split 03	0.7368	0.8571	0.6667
Split 04	0.8947	0.7143	1.0000
Split 05	1.0000	1.0000	1.0000
Overall	0.8602	0.8000	0.8966

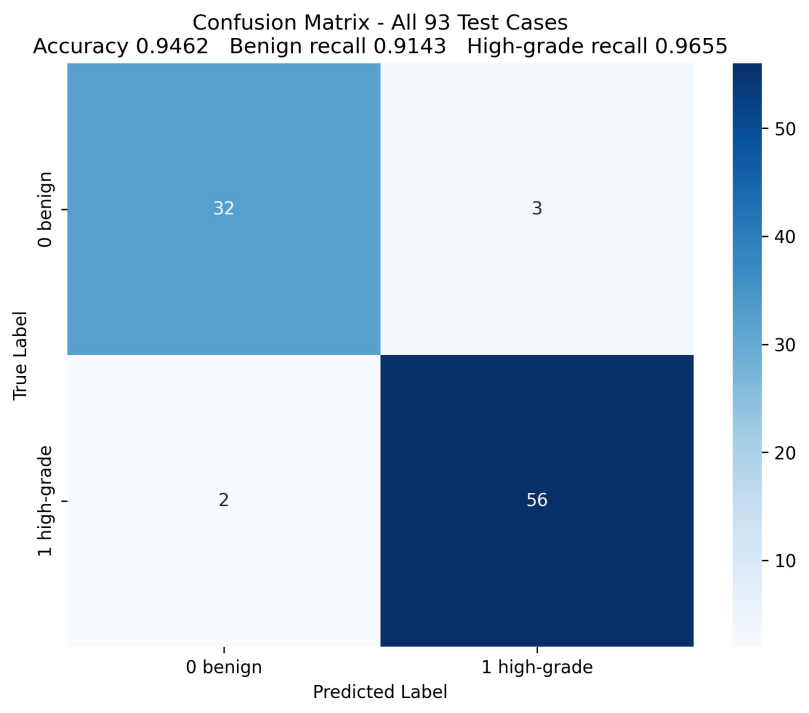
Uni2	Accuracy	Benign recall	High-grade recall
Split 01	0.8333	0.8571	0.8182
Split 02	0.9474	0.8571	1.0000
Split 03	1.0000	1.0000	1.0000
Split 04	0.9474	0.8571	1.0000
Split 05	1.0000	1.0000	1.0000
Overall	0.9462	0.9143	0.9655

Uni2 has better overall performance: +8.6% on accuracy, +11.43% on benign recall, +6.89% on High-grade recall

Jingyu

KimiaNet vs UNI2 embedding



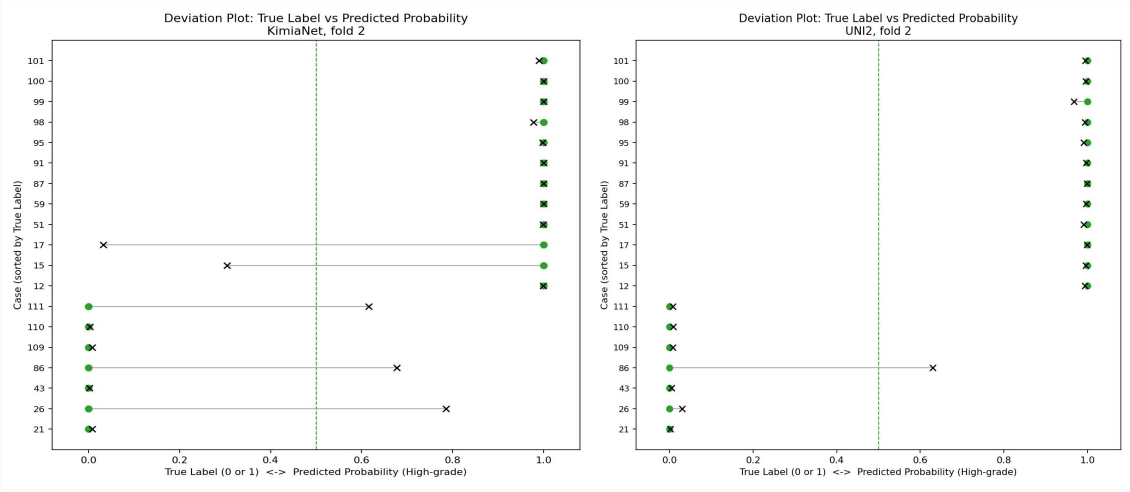


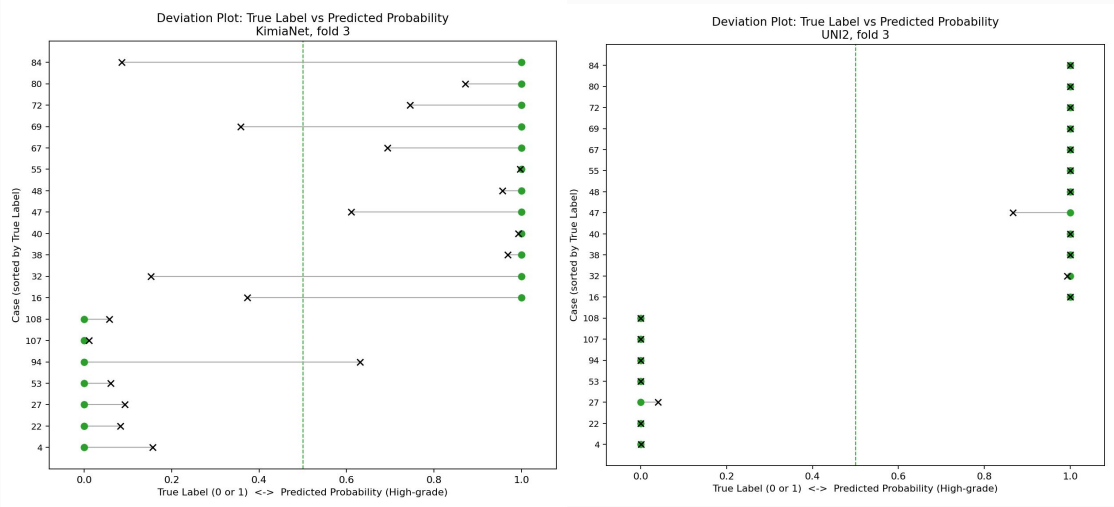
Jingyu

8 more correct prediction, 4 less false positive and 4 less false negative

Fold 2 & 3

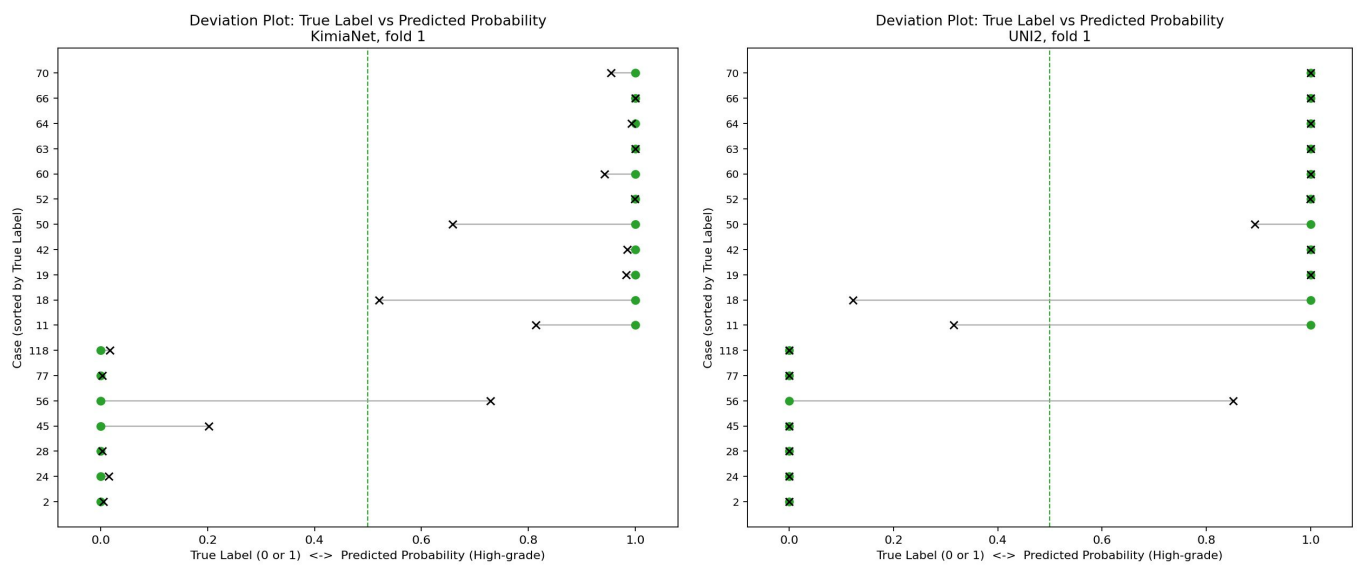
- The overall improvement is mainly driven by folds 2 and 3.
- Across these two folds, UNI2 correctly classifies 6 additional high-grade cases and 3 additional benign cases.
- UNI2 generally assigns higher probabilities to the correct class.
- Case 86 remains misclassified by both models.





Jingyu

Fold 1

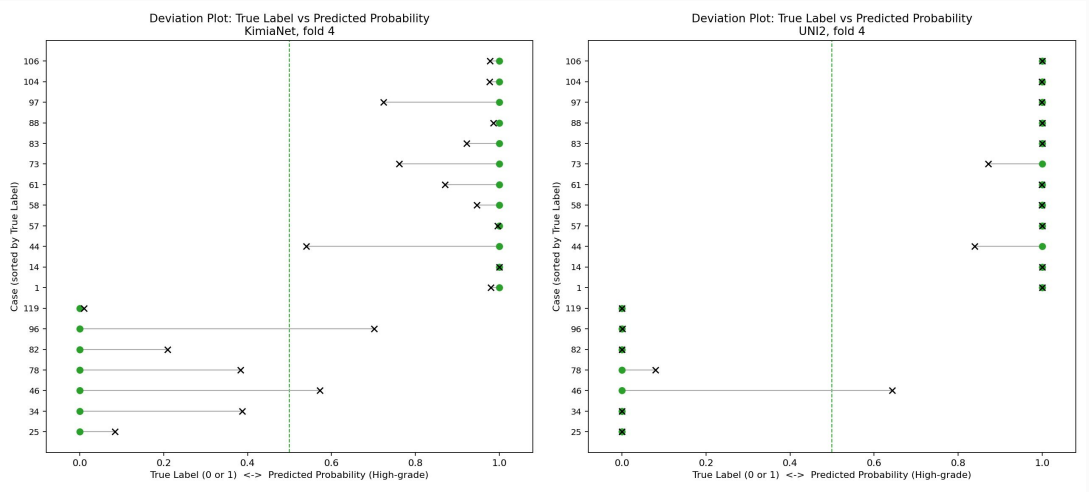


- **Cases 18 and 11:** Correctly classified as high-grade by KimiaNet, but misclassified as benign by UNI2.
- **Case 56:** Misclassified as high-grade by both models, with UNI2 assigning a higher probability to the incorrect class (≈0.85 vs. ≈0.73).

Jingyu

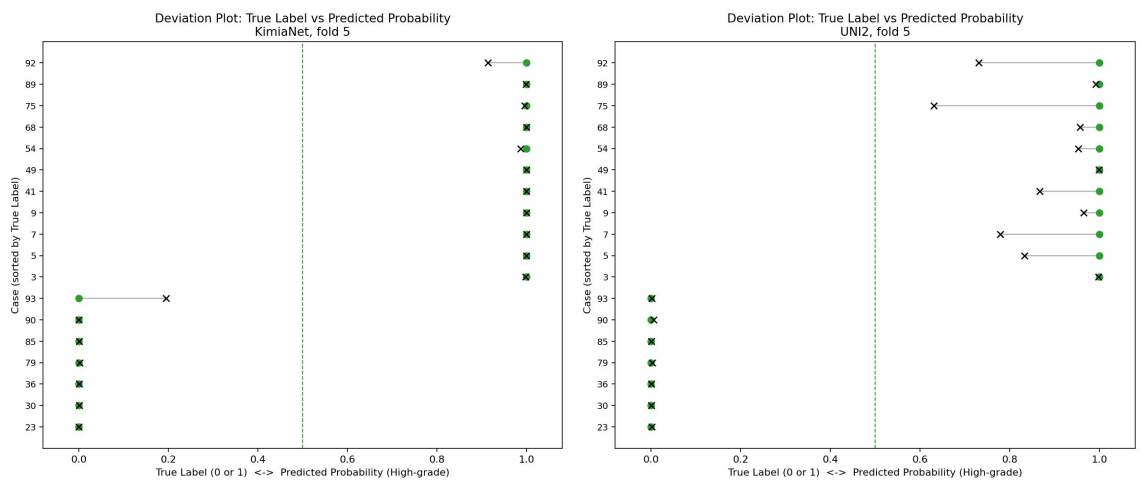
Fold 4

- UNI2 correctly classifies one additional benign case (case 96) and generally assigns higher probabilities to the correct class.
- Case 46 remains misclassified by both models.



Fold 5

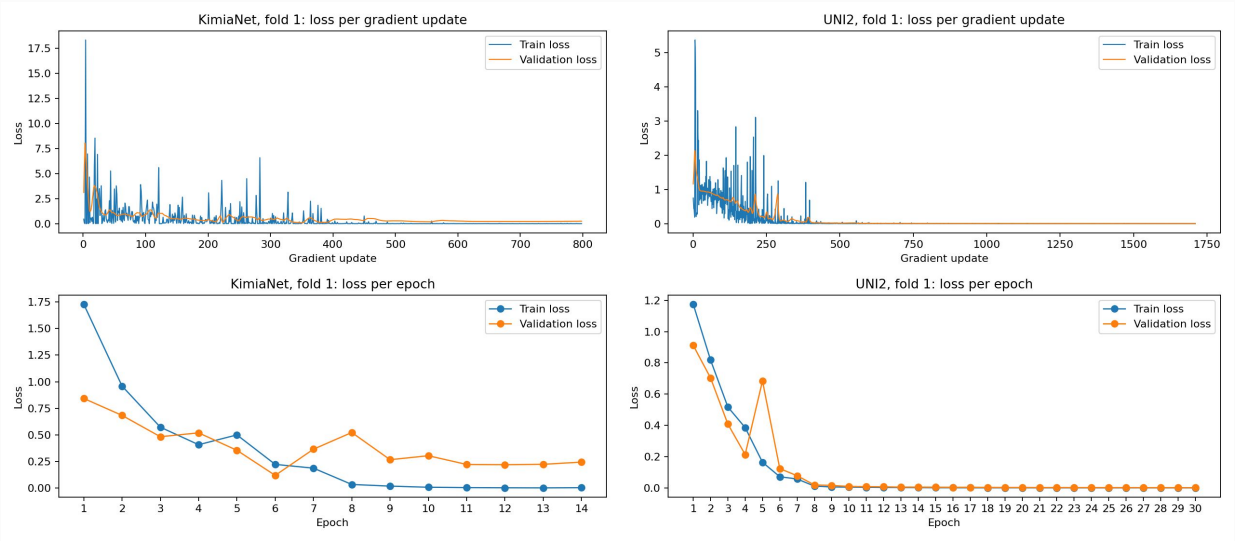
- Both models achieve 100% accuracy.
- UNI2 assigns lower high-grade probabilities to several high-grade cases, although all remain above the 0.5 threshold.



KimiaNet vs UNI2 Loss Curves

Fold 1

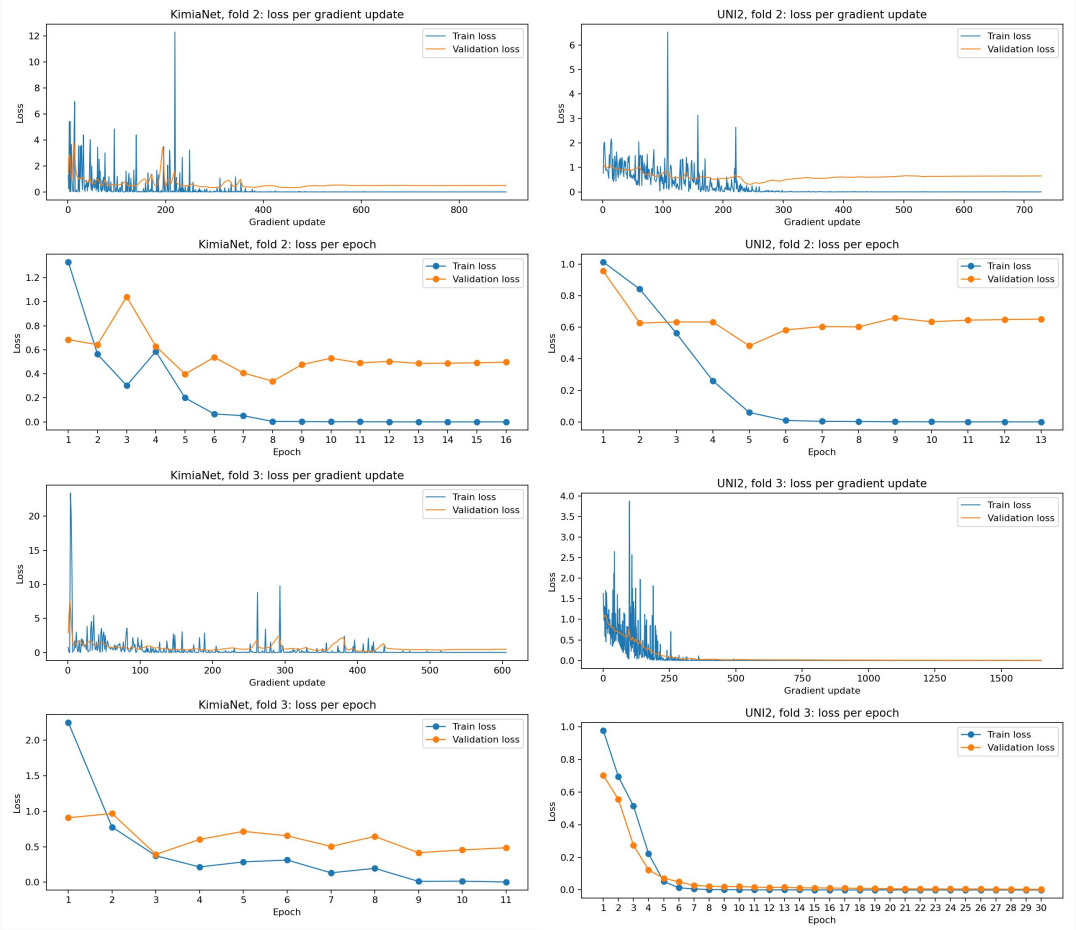
- **KimiaNet:** Training loss approaches zero, while validation loss rebounds after epoch 6, suggesting overfitting.
- **UNI2:** Training and validation losses both approach zero by approximately epoch 10, with a smaller gap.
- Both models show large fluctuations during early updates that diminish as training progresses.
- Despite lower validation loss, UNI2 achieves lower test accuracy in this fold (**83.33% vs. 94.44%**).



Loss Curves

Fold 2 and Fold 3

- **Fold 2:** Both models show a persistent gap between training and validation loss, indicating overfitting.
- **Fold 2:** UNI2 validation loss reaches its minimum around epoch 5, then increases as training loss continues to decrease.
- **Fold 3:** UNI2 training and validation losses both approach zero, while KimiaNet validation loss remains higher and fluctuates.
- UNI2 shows a clearer improvement in validation performance in **Fold 3 than in Fold 2**.

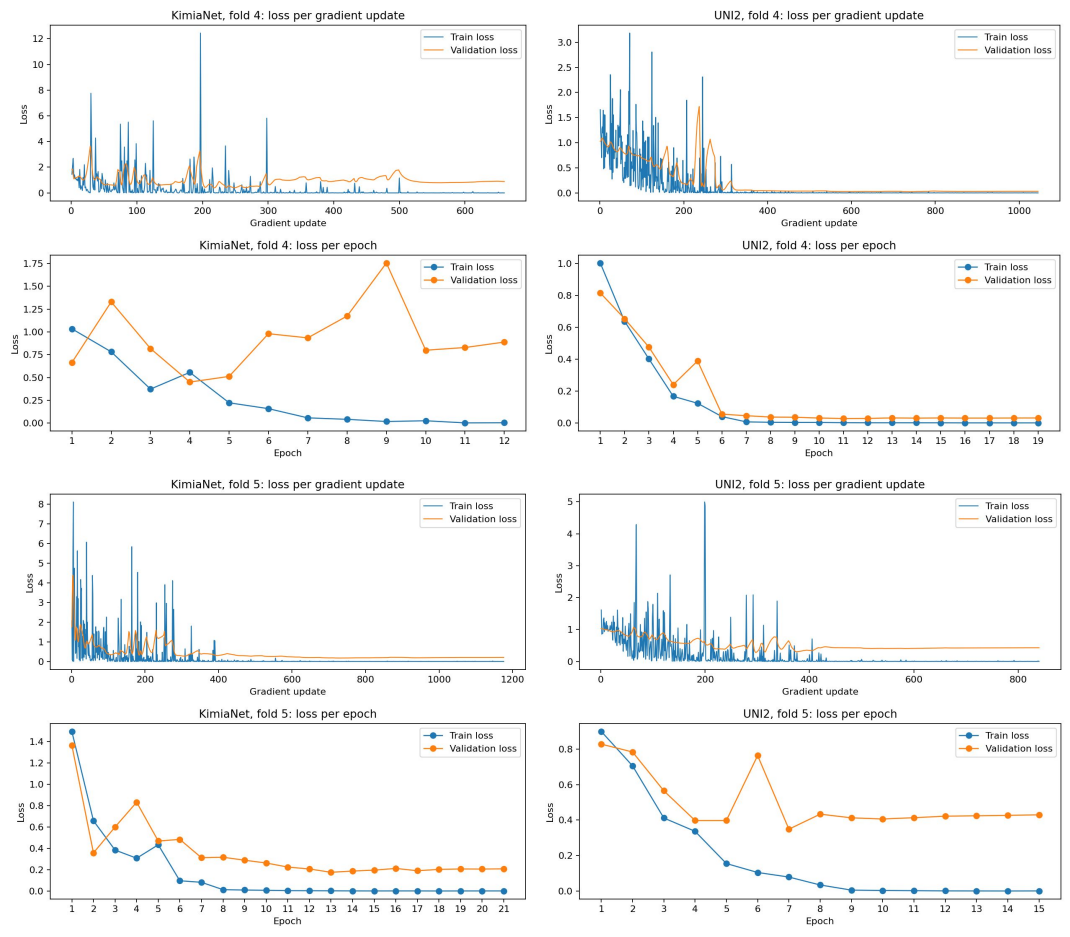


Yifan

Loss Curves

Fold 4 and Fold 5

- **Fold 4:** KimiaNet validation loss rises after epoch 4 despite decreasing training loss, showing clear signs of overfitting.
- **Fold 4:** UNI2 maintains low validation loss and a smaller gap between training and validation.
- **Fold 5:** Both models show signs of overfitting. UNI2 ends with higher validation loss than KimiaNet (**approximately 0.4 vs. 0.2**).
- Both models achieve **100% test accuracy in Fold 5**, despite their different validation losses.



KimiaNet vs UNI2 Conclusion and Reflection

Overall Performance:

- **UNI2 achieved higher pooled test accuracy: 94.62% vs. 86.02%, with improved recall for both classes.**
- Its advantage varied across folds, and some folds still showed signs of overfitting.

Possible Explanations:

- Large-scale self-supervised pretraining may help UNI2 learn more transferable pathology features.
- Pretraining on both **H&E and IHC** may better match our stains.
- These explanations remain hypotheses that require further testing.

Other Possible Encoders:

- **Virchow2 and H-optimus-0:** Alternative pathology image encoders for future comparison.
- **CONCH:** A pathology vision-language model whose image encoder can also provide features for MIL.

Mathlete

Pooling Redundancy

- DenseNet-121 halves the image 5 times: 2 steps round up, 3 round down, so the map side is:
  $n = \lfloor \lceil (\text{h or w}) / 4 \rceil / 8 \rfloor \approx (\text{h or w}) / 32$
- AdaptiveAvgPool(2 x 2) always outputs 4 quadrants, with side length deciding where a patch lands:

Patch Side Length	Map *n*	2 x 2 Output	% of Patches (83,630)
< 32px	N/A	Removed	2.40%
32-60 px	1	4 copies	4.47%
One side 32-60 px	1 x *n*	2 copies	0.017%
61-92, 125-156… px	Even #	4 unique quadrants	44.81%
93-124, 157-188, … px	Odd # != 1	Quadrants have 1 row of redundancy	48.3%

Ronan

Recommendation: Remove Redundancy

Case for removing :

- Only about half the patches have 4 independent quadrants
- The weights act per quadrant for ≈ 95% of patches, but as one summed matrix for the 4.5% that are small
- The 4096 -> 512 projector holds 84% of all trainable weights, so most of the model is working on a distinction in only half the data

Case for keeping:

- No information is lost
- Patches are aligned to the epithelium so for large patches the quadrants could carry epithelium/stroma layout, which possibly could carry some signal

Ronan

Recommendation: Remove Redundancy

Options:

- Drop small patches: < 61px removes ≈ 6.88% of all patches, < 64px removes ≈ 7.38%; this loses thin epithelium which may not be spread evenly across the two classes

- Possible shortcut: patch side = 1.1 x epithelium width, and side length alone decides whether the vector is fully redundant, overlapping, or distinct, so the model can read thickness from the vector's structure instead of needing to learn visual differences in morphology from the stain image

- **Proposed:** Global average pool keeps every patch, no duplication projector 2,097,664 ->524,800; trainable total 2,493,061 -> 920,197

Ronan

Dropout and Projection

Dropout location (currently: after the projector and before the classifier; none in attention)

- Per-patch masks largely average out after attention pooling
- Optuna tunes patch_proj_dropout and classifier_dropout separately, but models.py applies one shared value from config.py, so add both tuned dropout rates to config.py
- Add stain dropout: drop a whole stain before stain attention in training (keep ≥ 1), as in DuoHistoNet's branch dropout (p = 0.3); suits our cases with missing stains
- Bag-level augmentation on stored features: random slice choice instead of the first 5, and patch dropout

Projection length (currently 4096 -> 512)

- DuoHistoNet: 768 -> 512 with 15,000+ cases; we have ≈ 56 training cases with 1024-d input;
  Choose D ∈ {64, 128, 256, 512} by nested CV:
  D = 512: 920,197 256: 460,677 128: 230,917 64: 116,037 params
- Keep attention hidden dim 128: Ilse et al. found 64 / 128 / 256 made little difference

Ronan

Tuning

Needs Tuning:

- Width D ∈ {64, 128, 256, 512} (if not too costly)
- The two current dropout rates and new stain-dropout
- Benign class weight
- Learning rate and weight decay

Doesn't need tuning:

- Global Average Pooling
- Optuna pooled_dim = 4096 hardcoded, change to 1024 to match the new model

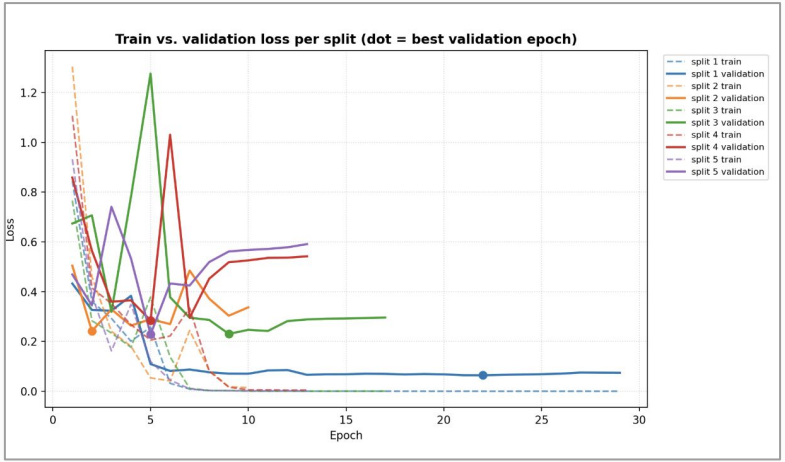
Other notes: optuna_training.py still uses CrossEntropyLoss with mean reduction, so we need to update that to use 'sum' as well, and another possible data augmentation is using color jitter to improve generalization, though this may not particularly help our model fit the current data

Ronan

Analyst

Presentation 1 Run Observations

Split	Best Epoch	Best Val Loss	Final Val Loss	Final / Best
Split 1	22	0.064	0.074	1.155
Split 2	2	0.241	0.337	1.397
Split 3	9	0.230	0.296	1.286
Split 4	5	0.284	0.542	1.908
Split 5	5	0.228	0.591	2.593

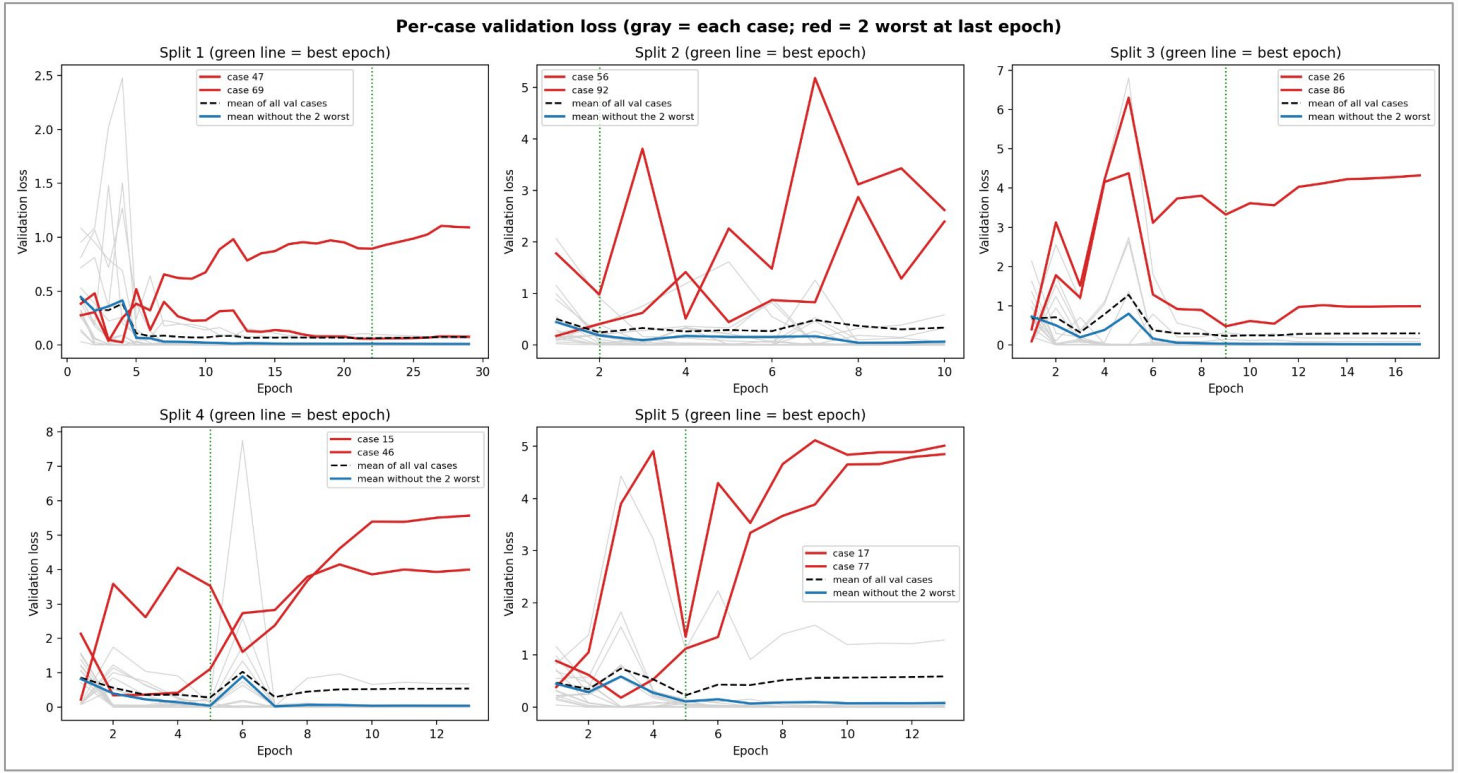


- 5 splits of the same 93 cases (35 benign and 58 high-grade). Each split trains on 3 folds, validates on 1, and tests on 1, with same model, hyperparameters, and seed in all five.
- Training loss ends in approximately 0 in every split. Final values range from -0.0006 to 0.014.
- Validation loss differs, as the best epoch ranges from 2 to 22. Final/best ranges from 1.2 to 2.6.
- The objective's example split (Team 2's split 5) used its own seed and folds, so it's a different set of cases than mine.
- Why does validation stay low in split 1 but rise in splits 2-5?

Taylor

https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/memorization_results.py
https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/results

Differences Between Splits



- Two cases carry 83% to 95% of each split's final validation loss.
- Accuracy is similar across splits: 16 to 18 correct out of 18 to 19 at the best epoch.
- Split 1: Worst case 47 ends at 34% high-grade probability with a loss of 1.1.
- Splits 2-5: The worst case loss is 2.6-5.6.
- Split 1's other 17 cases average a loss of 0.014.
- Among the cases that recur as errors, only 47 is in split 1's validation fold.

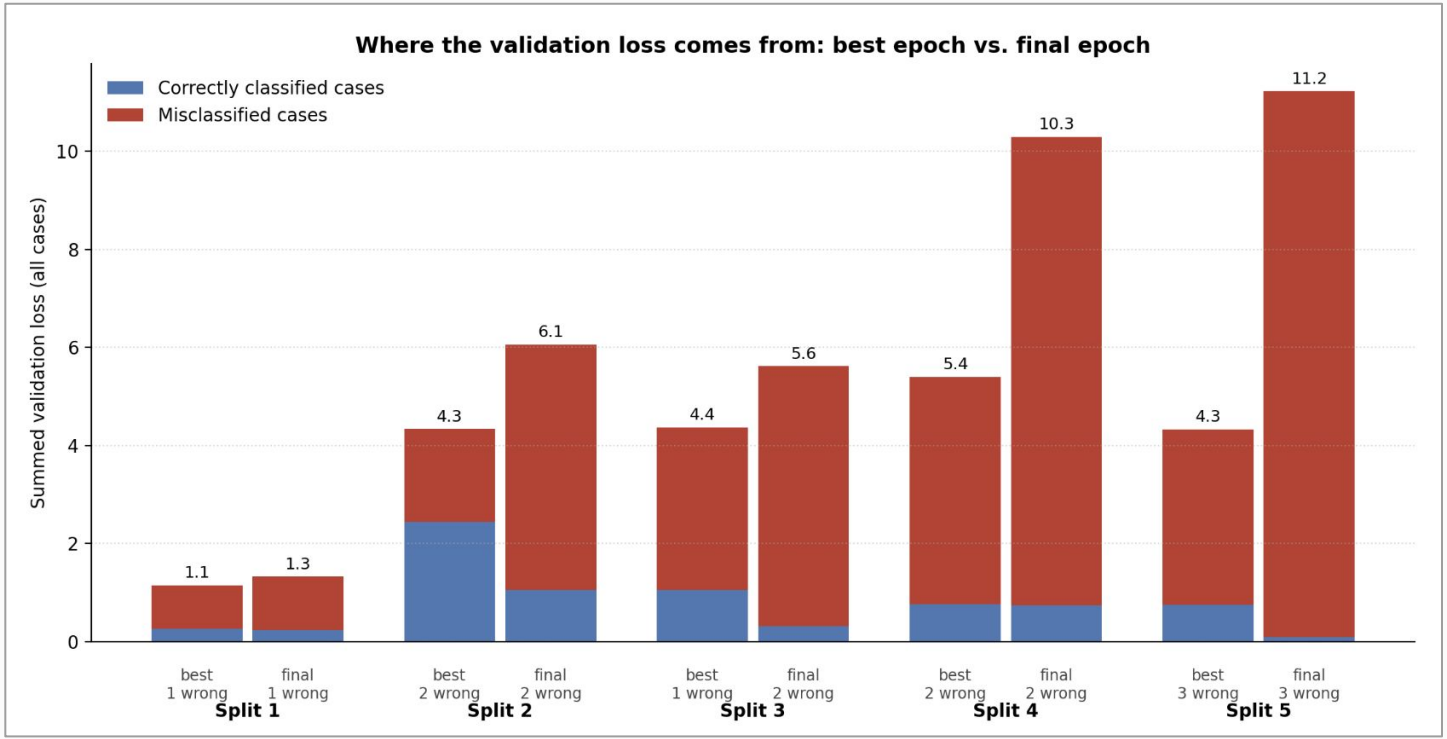
Taylor

https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/memorization_results.py
https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/results

Why the Validation Loss Rises

Note: Split 3's case 86 became misclassified, so accuracy fell from 18/19 at the best to 17/19 at the final epoch.

- Method: Divide each split's summed validation loss into misclassified and correctly classified at the best and last epoch.
- Misclassified cases' loss rises in every split.
- Correctly classified cases' loss falls or stays flat.
- In split 4, the same two cases (46 and 15) account for the whole rise in total loss.
- **Hypothesis**: Memorizing pushes output probabilities toward 0 and 1, so every remaining miss costs a lot.



Taylor

https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/memorization_results.py
https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/results

Potential Driving Factors

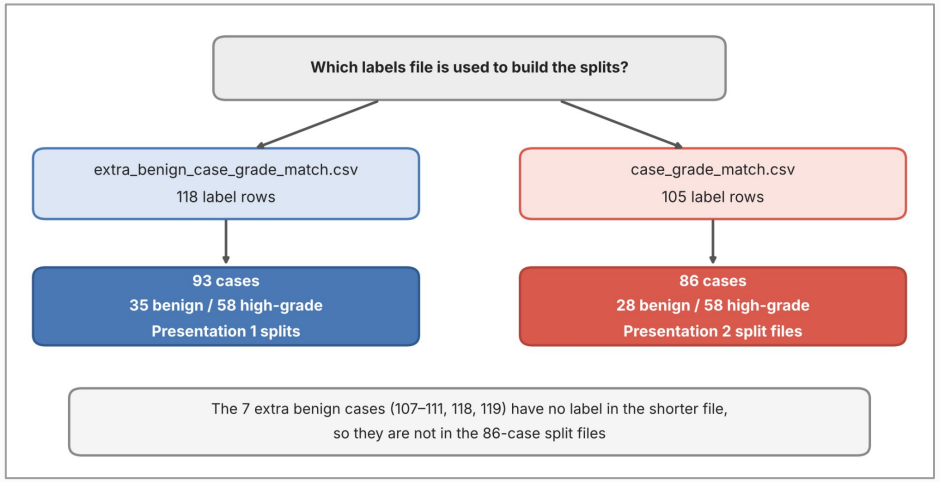
Case	Class	Patches used	Presentation 1 Val	Presentation 1 Test	Presentation 2 Val
26	Benign	868	Incorrect	Incorrect	Incorrect
56	Benign	424	Incorrect	Incorrect	Incorrect
46	Benign	185	Incorrect	Incorrect	Correct
45	Benign	123	Correct	Incorrect	Incorrect
86	Benign	1,266	Incorrect	Correct	Incorrect
94	Benign	678	Incorrect	Correct	Incorrect
32	High-Grade	399	Correct	Incorrect	Incorrect
47	High-Grade	946	Incorrect	Correct	Incorrect
92	High-Grade	460	Incorrect	Correct	Incorrect

- Cases 26 and 56 (both benign) are called high-grade in all 3 assessments.
- Seven more cases are wrong in two of three, and six of the nine recurring errors are benign cases called high-grade.
- **Outside check**: Team 2's run (different seed and folds) also misclassified 7 of these 9 cases, including case 56 at 99.997% confidence.
- Case size doesn't explain it, as these cases use 123 to 1,266 patches.
- Hypothesis: atypical benign cases or label noise (expert labels aren't 100% reliable). Could also be overconfidence after memorization.

Taylor

https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/memorization_results.py
https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/results

Presentation 2 Comparison



- **Important Observation:** The presentation 2 split files contain 86 cases, while my presentation 1 runs used 93.
  - The data for presentation 2 has 28 benign and 58 high-grade cases in total, while presentation 1's code has 35 benign, which changes the class balance.
  - Presentation 1 base code recently copied from the instructional team holds the 86-case split files, but my original version from presentation 1 holds 93 cases and the code walkthrough mentions 93.

Other Observations:

- What changed in presentation 2: Benign cases weighted ×2.5285 in the loss, the best-validation checkpoint is used for testing, and splits depend on the seed.
- Presentation 2's validation accuracy at the last epoch: 12/17, 14/17, 15/18, 17/17, and 16/17.
  - Cases 56 and 26 are still incorrect. Splits 4 and 5 look fine, but can't be attributed to the code changes alone, because the data also changed.
- Presentation 2's losses are weighted, so they're also not entirely comparable with presentation 1's. Case 56's final loss is 22.1 weighted vs. 8.7 unweighted.
- Proposed control could be to run presentation 2's code on presentation 1's 93-case split files.

Taylor

https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/memorization_results.py
https://github.com/arvindkrishna87/Team3_DivideNConquer/tree/main/presentation_2/Code1_AnalyzeResults_5Nov_TaylorMassey/results

Answer, Limitations, Next Steps

Answer

- All five splits memorize.
- Splits that look fine have validation folds without confident errors. Split 1's only error is borderline.
- The rise comes from 1 to 3 misclassified cases becoming more confident while typical cases improve. Another team's run flags 7 of the same 9 cases.
- Potential future fixes: label smoothing, choosing the checkpoint by accuracy or median loss, or reviewing labels.

Limitations

- 5 splits with one seed each, as well as 17-19 validation cases.
- Assessments use last-epoch models that share training data, so they aren't fully independent.
- Presentation 2's code ran on 86 cases.

Next Steps

- Potential pathologist review of cases 56 and 26.
- Confirm with the instructional team which split files are intended for each presentation.
- Retrain splits 4 and 5 with label smoothing and compare the losses of cases 46, 15, 77, 17, and 94.
- Retrain split 4 with 2-3 more seeds to see whether the same cases stay wrong.